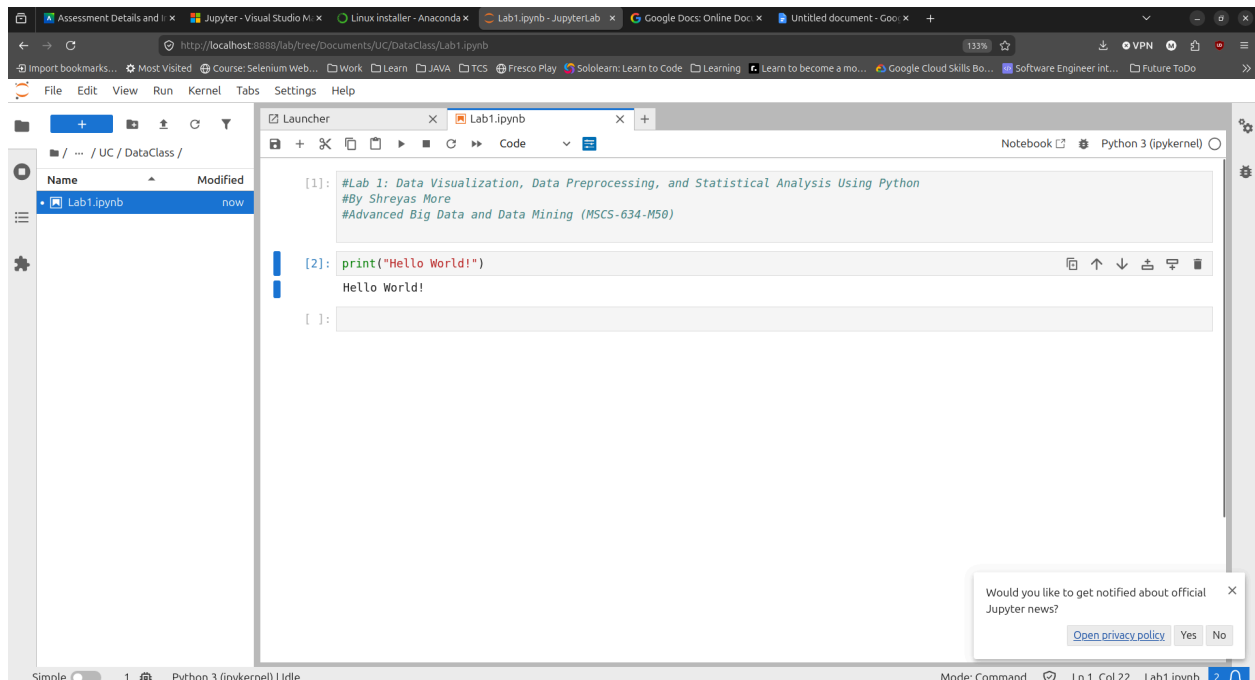
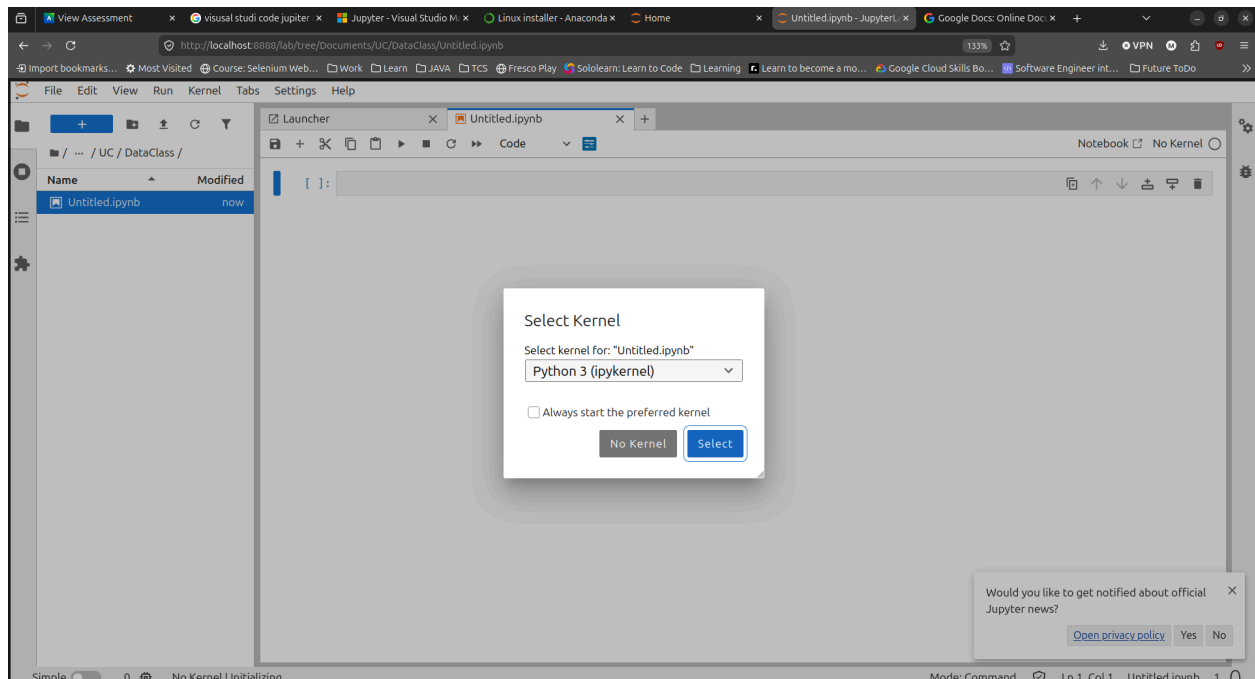


Created Jupyter Notebook



Minecraft Dataset

About this file

- biomes.csv**
Contains Minecraft biome data including name, category, temperature, precipitation status, dimension type, and color mapping.
- blocks.csv**
Contains block types used in Minecraft including material properties and structural attributes.
- items.csv**
Contains in-game items including tools, resources, and consumables used in gameplay.
- crafting_recipes.csv**
Defines crafting relationships between items, including inputs and resulting outputs.
- mobsp.csv**
Contains Minecraft entities including passive, neutral, and hostile mobs with their attributes.
- dimensions.csv**
Contains information about Minecraft worlds including Overworld, Nether, and End.
- enchantments.csv**
Contains enchantment types and their effects applied to tools, weapons, and armor.

biome_id	name	display_name	category	temperature	has_precipitation
0	badlands	Badlands	mesa	2.0	False
1	bamboo_jungle	Bamboo Jungle	jungle	0.95	True
2	basalt_deltas	Basalt Deltas	nether	3.0	False

Summary

- 8 files
- 67 columns

Choose the Minecraft dataset from Kaggle.

Files

Home / Documents / UC / DataClass

Name	Size	Modified
biomes.csv	4.1 kb	6 Jul 2026
blocks.csv	110.2 kb	6 Jul 2026
crafting_recipes.csv	88.2 kb	6 Jul 2026
dimensions.csv	272 bytes	6 Jul 2026
enchantments.csv	3.0 kb	6 Jul 2026
items.csv	59.3 kb	6 Jul 2026
Lab1.ipynb	2.7 kb	Today 20:14
mobsp.csv	4.9 kb	6 Jul 2026
status_effects.csv	1.2 kb	6 Jul 2026

The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer lists several CSV files, with 'Lab1.ipynb' selected. The code editor contains the following code:

```
[1]: #Lab 1: Data Preprocessing, Data Preprocessing, and Statistical Analysis Using Python
#By Shreyas More
#Advanced Big Data and Data Mining (MSCS-634-M50)

[2]: print("Hello World!")
Hello World!

[9]: import pandas as pd

df = pd.read_csv("mobs.csv")

df.head()
```

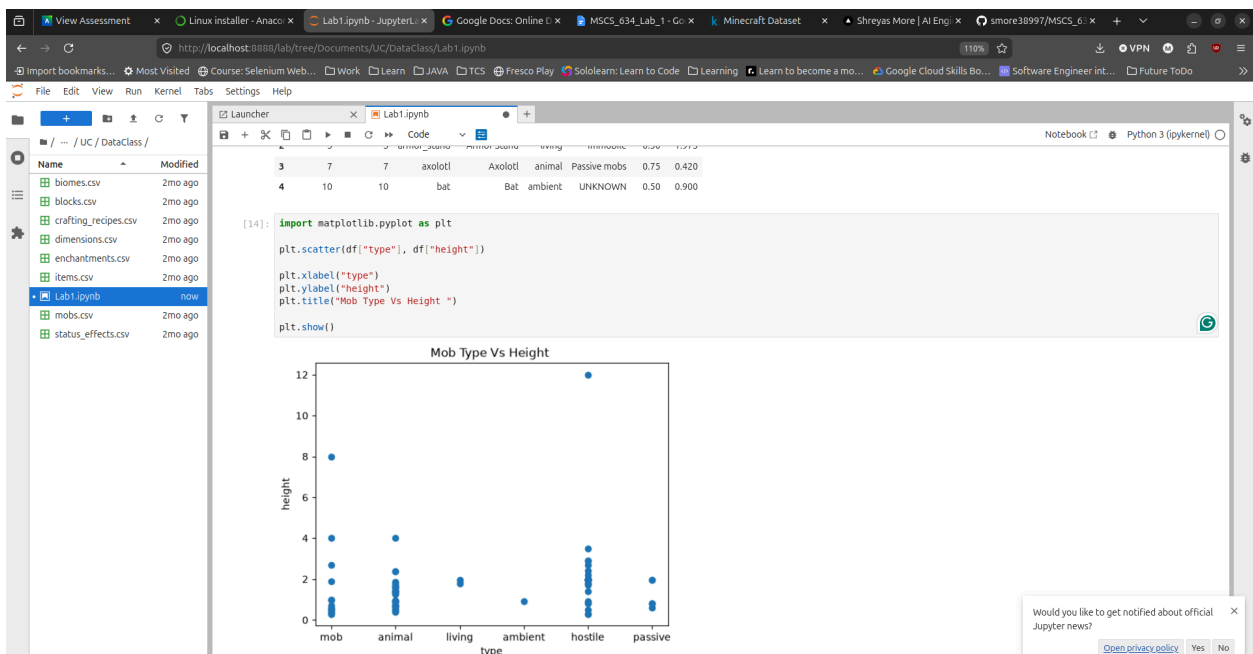
The output of the `df.head()` command is displayed as a table with 9 columns: `entity_id`, `internal_id`, `name`, `display_name`, `type`, `category`, `width`, and `height`. The first 5 rows of the data are shown:

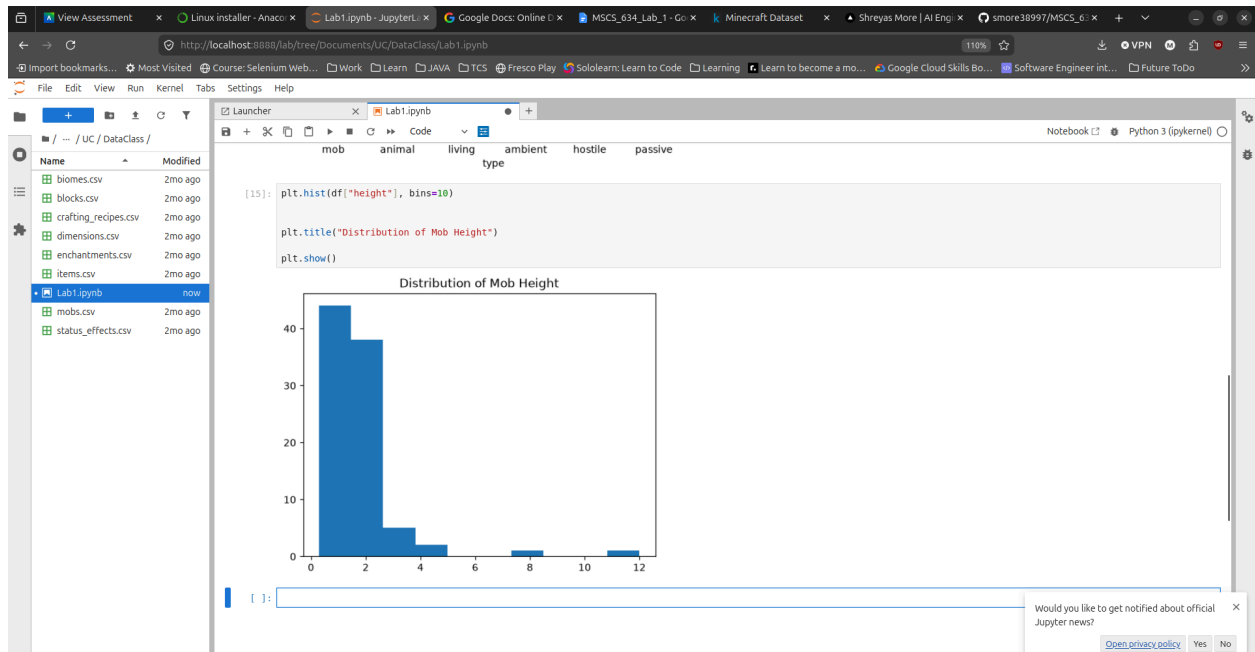
	entity_id	internal_id	name	display_name	type	category	width	height
0	2	2	allay	Allay	mob	Passive mobs	0.35	0.600
1	4	4	armadillo	Armadillo	animal	Passive mobs	0.70	0.650
2	5	5	armor_stand	Armor Stand	living	Immobile	0.50	1.975
3	7	7	axolotl	Axolotl	animal	Passive mobs	0.75	0.420
4	10	10	bat	Bat	ambient	UNKNOWN	0.50	0.900

A notification box at the bottom right asks: "Would you like to get notified about official Jupyter news?" with links for "Open privacy policy", "Yes", and "No".

`df.head()` > Displays the first 5 rows of our data `mobs.csv`

Mob Type vs Mob Height Scatter Plot. This type of visualization helps use see the patterns when it comes to Mob who are hostile in the game and their height.





Mob Height Histogram

This particular visualization helps us see the trends of height when compared to itself.

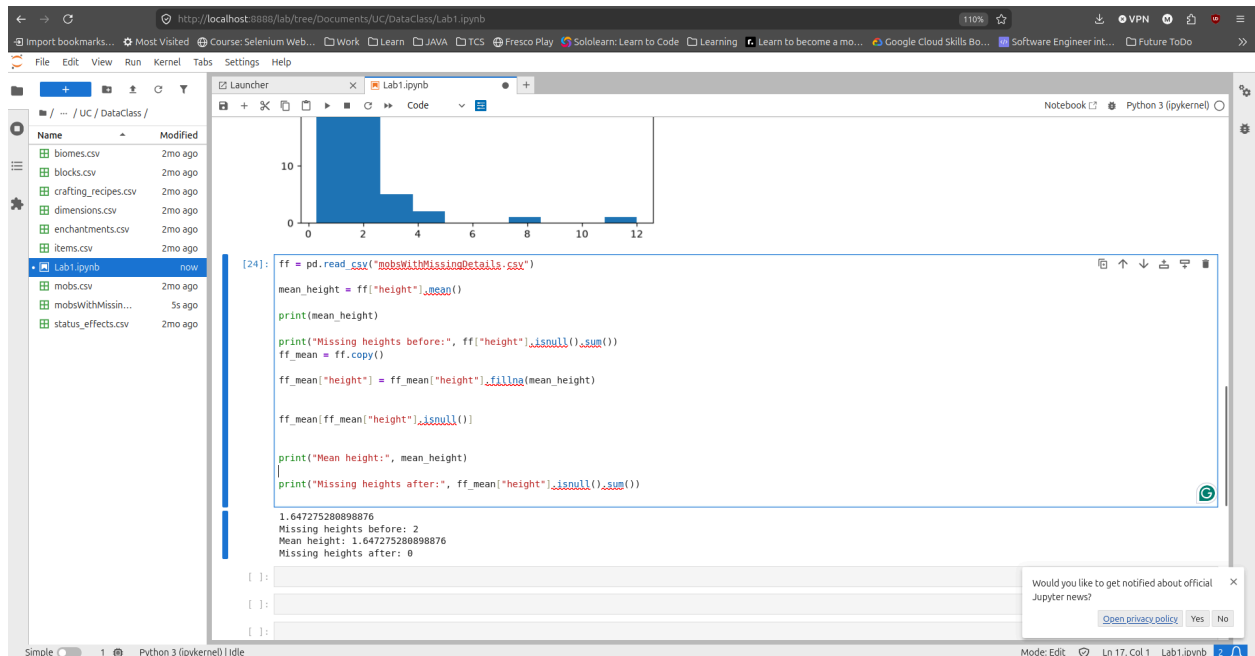
Mobs.csv before data deletion

mobsWithMissingDetails.csv — LibreOffice Calc									
File Edit View Insert Format Styles Sheet Data Tools Window Help									
Liberation Sans 10 pt 0.7									
Help us make LibreOffice even better! Get involved									
entity_id	internal_id	name	display_name	type	category	width	height		
2	2	allay	Allay	mob	Passive mobs	0.35	0.6		
4	4	armadillo	Armadillo	animal	Passive mobs	0.7	0.65		
5	5	armor_stand	Armor Stand	living	Immobile	0.5	1.975		
7	7	axolotl	Axolotl	animal	Passive mobs	0.75	0.42		
10	10	bat	Bat	ambient	UNKNOWN	0.5	0.9		
11	11	bee	Bee	animal	Passive mobs	0.7	0.6		
14	14	blaze	Blaze	hostile	Hostile mobs	0.6	1.8		
16	16	bogged	Bogged	hostile	Hostile mobs	0.6	1.99		
17	17	breeze	Breeze	hostile	Hostile mobs	0.6	1.77		
19	19	camel	Camel	animal	Passive mobs	1.7	2.375		
20	20	camel_husk	Camel Husk	animal	Passive mobs	1.7	2.375		
21	21	cat	Cat	animal	Passive mobs	0.6	0.7		
22	22	cave_spider	Cave Spider	hostile	Hostile mobs	0.7	0.5		
26	26	chicken	Chicken	animal	Passive mobs	0.4	0.7		
27	27	cod	Cod	mob	Passive mobs	0.5	0.3		
28	28	copper_golem	Copper Golem	mob	Passive mobs	0.49	0.98		
30	30	cow	Cow	animal	Passive mobs	0.9	1.4		
31	31	creaking	Creaking	hostile	Hostile mobs	0.9	2.7		
32	32	creeper	Creeper	hostile	Hostile mobs	0.6	1.7		
35	35	dolphin	Dolphin	passive	Passive mobs	0.9	0.6		
36	36	donkey	Donkey	animal	Passive mobs	1.3964844	1.5		
38	38	drowned	Drowned	hostile	Hostile mobs	0.6	1.95		

entity_id	internal_id	name	display_name	type	category	width	height
2	2	allay	Allay	mob	Passive mobs	0.35	0.6
4	4	armadillo	Armadillo	animal	Passive mobs	0.7	0.65
5	5	armor_stand	Armor Stand	living	Immobile	0.5	1.975
7	7	axolotl	Axolotl	animal	Passive mobs	0.75	0.42
10	10	bat	Bat	ambient	UNKNOWN	0.5	0.9
11	11	bee	Bee	animal	Passive mobs	0.7	0.9
14	14	blaze	Blaze	hostile	Hostile mobs	0.6	1.8
16	16	bogged	Bogged	hostile	Hostile mobs	0.6	1.99
17	17	breeze	Breeze	hostile	Hostile mobs	0.6	1.77
19	19	camel	Camel	animal	Passive mobs	1.7	
20	20	camel_husk	Camel Husk	animal	Passive mobs	1.7	2.375
21	21	cat	Cat	animal	Passive mobs	0.6	0.7
22	22	cave_spider	Cave Spider	hostile	Hostile mobs	0.7	0.5
26	26	chicken	Chicken	animal	Passive mobs	0.4	0.7
27	27	cod	Cod	mob	Passive mobs	0.5	0.3
28	28	copper_golem	Copper Golem	mob	Passive mobs	0.49	0.98
30	30	cow	Cow	animal	Passive mobs	0.9	1.4
31	31	creaking	Creaking	hostile	Hostile mobs	0.9	2.7
32	32	creeper	Creepers	hostile	Hostile mobs	0.6	1.7
35	35	dolphin	Dolphin	passive	Passive mobs	0.9	0.6
36	36	donkey	Donkey	animal	Passive mobs	1.3964844	1.5
38	38	drowned	Drowned	hostile	Hostile mobs	0.6	1.95

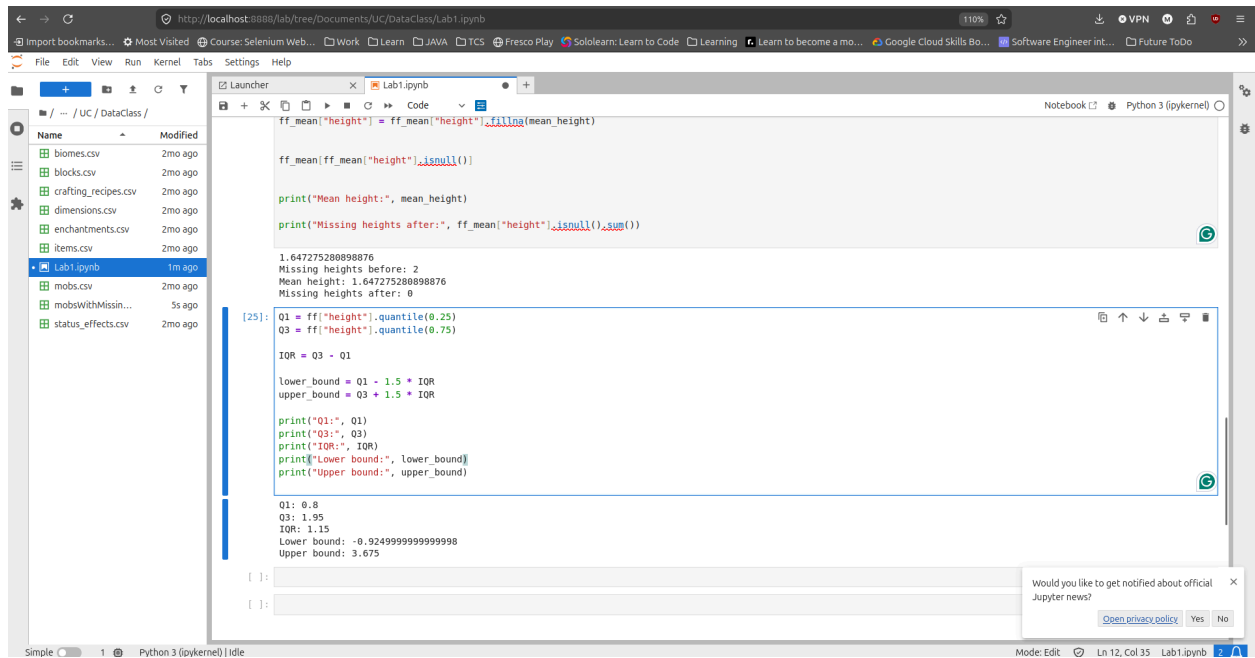
Mobs.csv with some data deletion.

For missing height is most appropriate to use `mean_height = df["height"].mean()`



Mean Replacement Method to deal with missing details.

IQR Method to determine outliers.



```
ff_mean["height"] = ff_mean["height"].fillna(mean_height)

ff_mean[ff_mean["height"].isnull()]

print("Mean height:", mean_height)

print("Missing heights after:", ff_mean["height"].isnull().sum())

1.647275280898876
Missing heights before: 2
Mean height: 1.647275280898876
Missing heights after: 0

[25]: Q1 = ff["height"].quantile(0.25)
      Q3 = ff["height"].quantile(0.75)

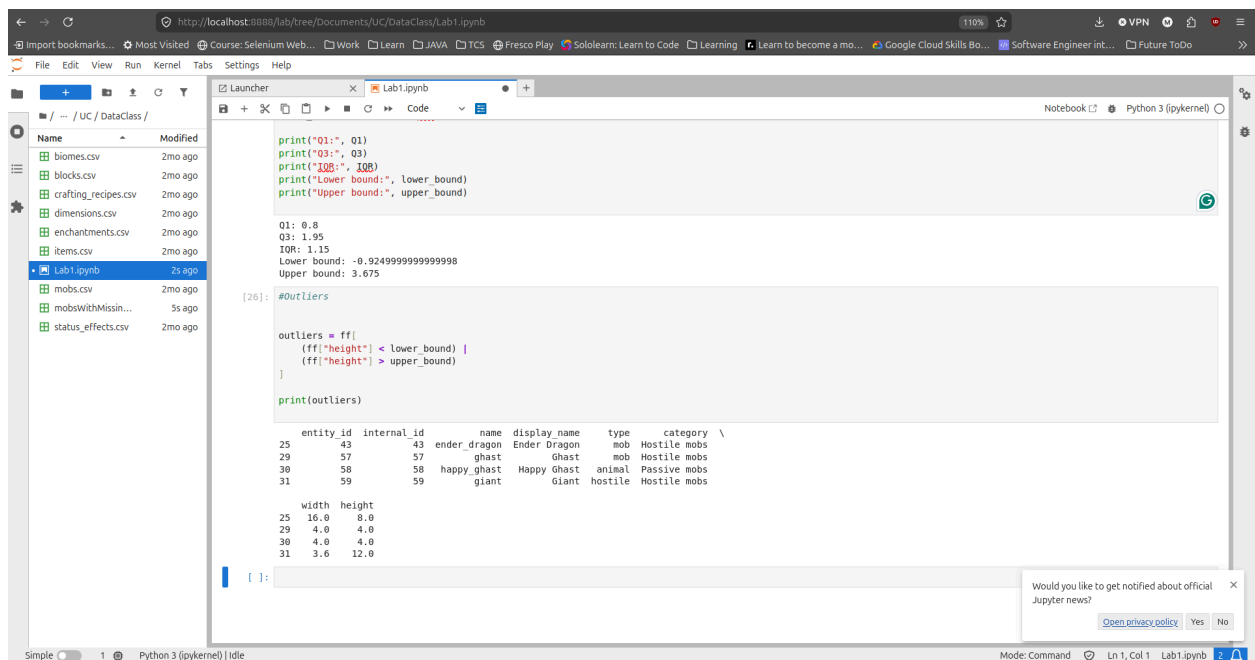
      IQR = Q3 - Q1

      lower_bound = Q1 - 1.5 * IQR
      upper_bound = Q3 + 1.5 * IQR

      print("Q1:", Q1)
      print("Q3:", Q3)
      print("IQR:", IQR)
      print("Lower bound:", lower_bound)
      print("Upper bound:", upper_bound)

      Q1: 0.8
      Q3: 1.95
      IQR: 1.15
      Lower bound: -0.9249999999999998
      Upper bound: 3.675

[ ]:
[ ]:
```



```
print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)

Q1: 0.8
Q3: 1.95
IQR: 1.15
Lower bound: -0.9249999999999998
Upper bound: 3.675

[26]: #Outliers

outliers = ff[
    (ff["height"] < lower_bound) |
    (ff["height"] > upper_bound)
]

print(outliers)

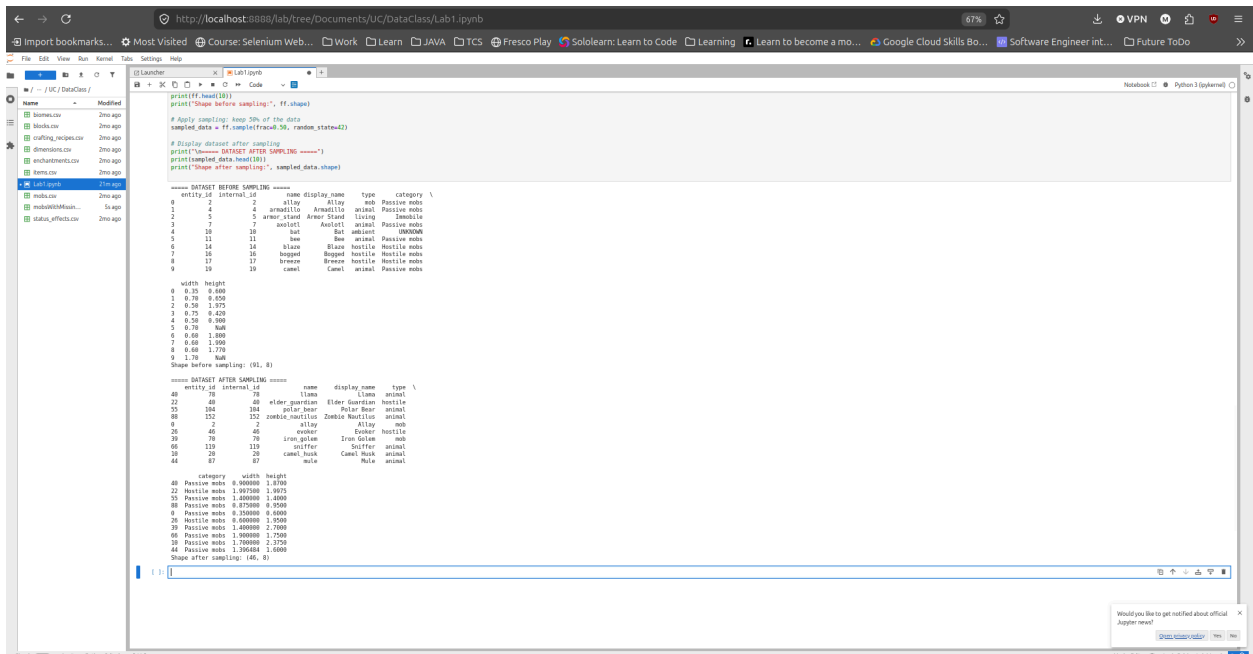
entity_id  internal_id  name  display_name  type  category \
25         43         43  ender_dragon  mob  Hostile mobs
29         57         57    happy_ghast  mob  Hostile mobs
30         58         58    happy_ghast  animal  Passive mobs
31         59         59    happy_ghast  hostile  Hostile mobs

width  height
25    16.0    8.0
29     4.0    4.0
30     4.0    4.0
31     3.6   12.0

[ ]:
```

The Outliers makes sense because the Ender Dragon is one of the biggest mobs in Minecraft.

Data Reduction: Before and After Screenshot



```
import sklearn.preprocessing
import train_test_split

# Load dataset
ff = pd.read_csv('modestMushrooms.csv')

# Select the numerical column
scalar = MinMaxScaler()

ff['height_scaled'] = scalar.fit_transform(ff['height'])

print('===== MIN-MAX SCALING =====')
print(ff[['height', 'height_scaled']].head(10))

# Divide height into meaningful categories
ff['height_category'] = pd.cut(
    ff['height'],
    bins=[0, 150, 300, float('inf')],
    labels=['Short', 'Medium', 'Tall'])

print('===== DISCRETIZATION =====')
print(ff[['height', 'height_category']].head(10))

# End Result

print('===== FINAL DATASET =====')
print(ff[['height', 'height_scaled', 'height_category']].head(10))

===== DATASET BEFORE SAMPLING =====
entity_id internal_id name display_name type category \
0 2 2 alloy Alloy mob Passive eds
1 4 4 armoillic Armoredillo animal Passive eds
2 5 5 armor_stand Armor Stand Living Inedible
3 7 7 bowtact Bow animal Passive eds
5 15 15 bat Bat animal UNKNOWN
6 16 16 bow Bow animal Passive eds
7 16 16 bopped Bopped hostile hostile eds
8 17 17 brwewe Brwewe hostile hostile eds
9 19 19 camel Camel animal Passive eds

width height
0 0.25 0.500
1 0.25 0.500
2 0.50 1.075
3 0.25 0.420
4 0.50 0.500
5 0.25 0.500
6 0.50 1.000
7 0.50 0.490
8 0.50 1.070
9 1.25 NaN

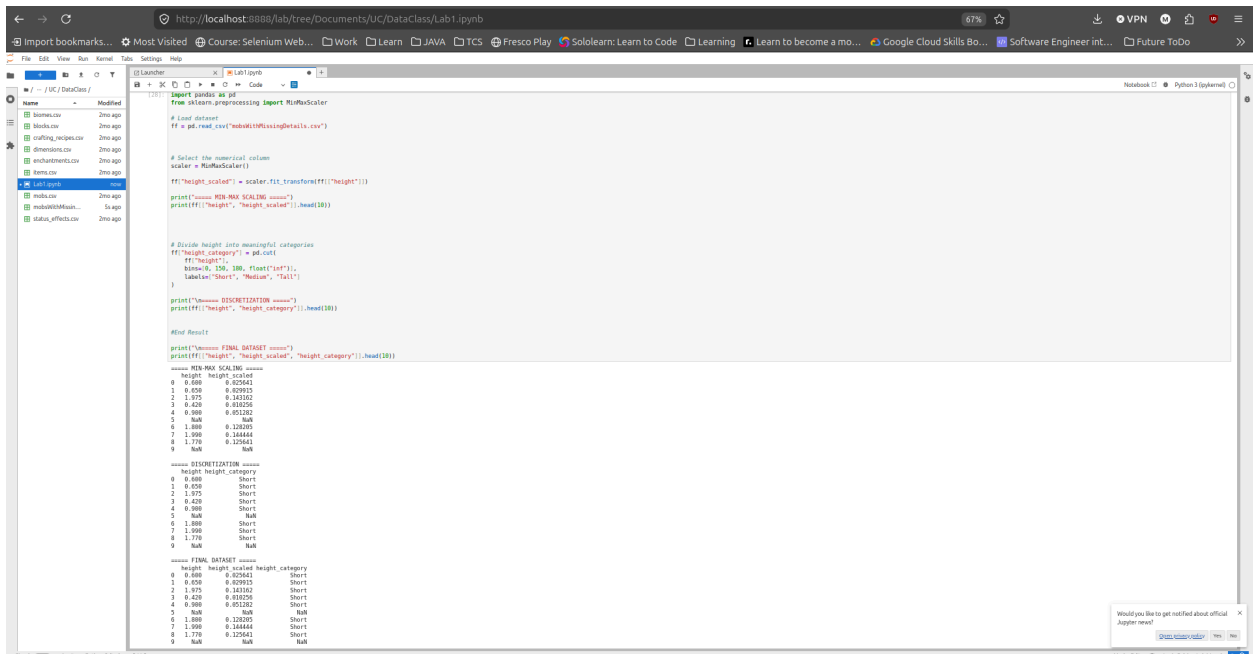
Shape before sampling: (91, 8)

===== DATASET AFTER SAMPLING =====
entity_id internal_id name display_name type category \
48 78 78 llama llama animal
22 40 40 elder_guardian Elder Guardian hostile
55 104 104 polar_bear Polar Bear animal
88 152 152 zombie_monster Zombie Monster animal
20 40 40 alloy Alloy mob
26 40 40 bow Bow animal
66 119 119 iron_golem Iron Golem mob
66 119 119 sciffer Sciffer animal
39 20 20 camel_horn animal
44 87 87 mule Mule animal

category width height
48 Passive mobs 0.500000 1.4700
22 Hostile mobs 1.007000 1.0070
55 Passive mobs 1.400000 1.4000
88 Passive mobs 0.750000 0.9500
20 Passive mobs 0.750000 0.5000
26 Hostile mobs 0.500000 1.0000
39 Passive mobs 1.400000 2.7000
66 Passive mobs 1.500000 1.7000
66 Passive mobs 1.700000 2.2750
44 Passive mobs 1.250000 2.0000

Shape after sampling: (46, 8)
```

Data Scaling :



```
import sklearn.preprocessing

# Load dataset
ff = pd.read_csv('modestMushrooms.csv')

# Select the numerical column
scalar = MinMaxScaler()

ff['height_scaled'] = scalar.fit_transform(ff['height'])

print('===== MIN-MAX SCALING =====')
print(ff[['height', 'height_scaled']].head(10))

# Divide height into meaningful categories
ff['height_category'] = pd.cut(
    ff['height'],
    bins=[0, 150, 300, float('inf')],
    labels=['Short', 'Medium', 'Tall'])

print('===== DISCRETIZATION =====')
print(ff[['height', 'height_category']].head(10))

# End Result

print('===== FINAL DATASET =====')
print(ff[['height', 'height_scaled', 'height_category']].head(10))

===== MIN-MAX SCALING =====
height height_scaled
0 0.500 0.500000
1 0.500 0.500000
2 1.075 0.542500
3 0.420 0.002500
4 0.500 0.500000
5 NaN NaN
6 1.000 0.520000
7 1.000 0.544444
8 1.070 0.525000
9 NaN NaN

===== DISCRETIZATION =====
height height_category
0 0.500 Short
1 0.500 Short
2 1.075 Short
3 0.420 Short
4 0.500 Short
5 NaN NaN
6 1.000 Short
7 1.000 Short
8 1.070 Short
9 NaN NaN

===== FINAL DATASET =====
height height_scaled height_category
0 0.500 0.500000 Short
1 0.500 0.500000 Short
2 1.075 0.542500 Short
3 0.420 0.002500 Short
4 0.500 0.500000 Short
5 NaN NaN
6 1.000 0.520000 Short
7 1.000 0.544444 Short
8 1.070 0.525000 Short
9 NaN NaN
```

The screenshot shows a JupyterLab environment with a file browser on the left, a central workspace with a Jupyter Notebook, and a right sidebar with tabs for 'Launcher', 'Lab1.ipynb', and 'Code'. The notebook contains the following code and output:

```
##### FINAL DATASET #####
height scaled height category
0 0.688 0.025641 Short
1 0.659 0.029315 Short
2 1.975 0.143162 Short
3 0.426 0.018256 Short
4 0.988 0.051282 Short
5 NaN NaN NaN
6 1.888 0.128285 Short
7 1.998 0.144444 Short
8 1.778 0.125641 Short
9 NaN NaN NaN
```

```
[29]: df.info
```

```
[29]: <bound method DataFrame.info of
0      2      2      allay      Allay      mob
1      4      4      armadillo      Armadillo      animal
2      5      5      armor_stand      Armor Stand      living
3      7      7      axolotl      Axolotl      animal
4     10     10      bat      Bat      ambient
...
86    150    150      zombie      Zombie      hostile
87    151    151  zombie_horse  Zombie Horse      animal
88    152    152  zombie_nurtius  Zombie Nurtius      animal
89    153    153  zombie_villager  Zombie Villager      hostile
90    154    154  zombieified_piglin  Zombieified Piglin      hostile
```

```
category      width  height
0  Passive mobs  0.350000  0.688
1  Passive mobs  0.700000  0.658
2  Immobile     0.500000  1.975
3  Passive mobs  0.750000  0.426
4  UNKNOWN     0.500000  0.988
...
86  Hostile mobs  0.600000  1.950
87  Passive mobs  1.396484  1.688
88  Passive mobs  0.875000  0.950
89  Hostile mobs  0.600000  1.950
90  Hostile mobs  0.600000  1.950
```

```
[91 rows x 8 columns]
```

The screenshot shows a Jupyter Notebook interface with a browser window. The address bar indicates the notebook is running on a local host. The notebook has two tabs: 'Launcher' and 'Lab1.ipynb'. The 'Lab1.ipynb' tab is active, showing a code cell with the following content:

```
90      154      154      zombified_piglin      Zombified Piglin      hostile
```

The output of the code cell is a table with 91 rows and 8 columns. The first few rows are:

	category	width	height
0	Passive mobs	0.350000	0.600
1	Passive mobs	0.700000	0.650
2	Immobil	0.500000	1.975
3	Passive mobs	0.750000	0.420
4	UNKNOWN	0.500000	0.900

The output continues with more rows, including 'Hostile mobs' and 'Passive mobs'. The final row is:

	category	width	height
90	Hostile mobs	0.600000	1.950

Below the table, the output shows the dimensions of the DataFrame:

```
[91 rows x 8 columns]
```

The code cell then calls the `df.describe()` method, which returns a summary of the data. The output is:

```
[30]:
```

	entity_id	internal_id	width	height
count	91.000000	91.000000	91.000000	91.000000
mean	83.032967	83.032967	1.091790	1.643764
std	46.589329	46.589329	1.714224	1.526764
min	2.000000	2.000000	0.350000	0.300000
25%	40.500000	40.500000	0.600000	0.750000
50%	88.000000	88.000000	0.700000	1.600000
75%	122.500000	122.500000	0.900000	1.950000
max	154.000000	154.000000	16.000000	12.000000

The notebook interface also shows a 'Launcher' tab and a 'Code' tab. The 'Code' tab is active, showing the code cell. The 'Launcher' tab shows the notebook's file structure, including a 'Lab1.ipynb' file.

MEAN, MEDIAN, MODE, and Maximum

The screenshot shows a Jupyter Notebook interface with a browser window at the top displaying the URL `http://localhost:8888/lab/tree/Documents/UC/DataClass/Lab1.ipynb`. The notebook has a sidebar on the left with a file explorer and a list of cells. The main area shows a code cell with the following Python code:

```
[31]: import pandas as pd

# Load dataset
df = pd.read_csv("mobsWithMissingDetails.csv")

# Select height column and remove missing values
height = df["height"].dropna()

# Calculate central tendency measures
minimum = height.min()
maximum = height.max()
mean = height.mean()
median = height.median()
mode = height.mode()[0]

# Display results
print("==== CENTRAL TENDENCY MEASURES =====")
print("Minimum:", minimum)
print("Maximum:", maximum)
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)

===== CENTRAL TENDENCY MEASURES =====
Minimum: 0.3
Maximum: 12.0
Mean: 1.6472752808988762
Median: 1.6
Mode: 1.95
```

The output of the code is displayed in a table format:

std	46.589329	46.589329	1.714224	1.526764
min	2.000000	2.000000	0.350000	0.300000
25%	40.500000	40.500000	0.600000	0.750000
50%	88.000000	88.000000	0.700000	1.600000
75%	122.500000	122.500000	0.900000	1.950000
max	154.000000	154.000000	16.000000	12.000000

Dispersion Measures:

The screenshot shows a Jupyter Notebook interface with a browser window at the top displaying the URL `http://localhost:8888/lab/tree/Documents/UC/DataClass/Lab1.ipynb`. The notebook has a sidebar on the left with a file explorer and a list of cells. The main area shows a code cell with the following Python code:

```
# Select height and remove missing values
height = ff["height"].dropna()

# Calculate dispersion measures
minimum = height.min()
maximum = height.max()

data_range = maximum - minimum

q1 = height.quantile(0.25)
q2 = height.quantile(0.50)
q3 = height.quantile(0.75)

iqr = q3 - q1

variance = height.var()
standard_deviation = height.std()

# Display results
print("==== DISPERSION MEASURES =====")
print("Minimum:", minimum)
print("Maximum:", maximum)
print("Range:", data_range)

print("\nQuartiles:")
print("Q1 (25%):", q1)
print("Q2 (50%):", q2)
print("Q3 (75%):", q3)

print("\nInterquartile Range (IQR):", iqr)
print("Variance:", variance)
print("Standard Deviation:", standard_deviation)

===== DISPERSION MEASURES =====
Minimum: 0.3
Maximum: 12.0
Range: 11.7

Quartiles:
Q1 (25%): 0.8
Q2 (50%): 1.6
Q3 (75%): 1.95

Interquartile Range (IQR): 1.15
Variance: 2.365517562563841
Standard Deviation: 1.5380239148218213
```

Correlation Analysis:

← → ↻

http://localhost:8888/lab/tree/Documents/UC/DataClass/Lab1.ipynb

87% ☆

📶 VPN

🔌

🔌

🔌

🔌

🔍 Import bookmarks... ⚙️ Most Visited 📑 Course: Selenium Web... 📁 Work 📁 Learn 📁 JAVA 📁 TCS 📁 Fresco Play 📁 Sololearn: Learn to Code 📁 Learning 📁 Learn to become a mo... 📁 Google Cloud Skills Bo... 📁 Software Engineer int... 📁 Future ToDo >>

🔌 Tabs ⚙️ Settings 🆘 Help

🔌 Launcher 📄 Lab1.ipynb +

🔍 + 🔍 📄 📄 📄 📄 📄 Code ▾

===== DISPERSION MEASURES =====
Minimum: 0.3
Maximum: 12.0
Range: 11.7

Quartiles:
Q1 (25%): 0.8
Q2 (50%): 1.6
Q3 (75%): 1.95

Interquartile Range (IQR): 1.15
Variance: 2.365517562563841
Standard Deviation: 1.5380239148218213

[34]: `import pandas as pd`
`# Load dataset`
`ff = pd.read_csv("mobsWithMissingDetails.csv")`
`# Select numerical columns`
`numerical_data = ff.select_dtypes(include="number")`
`# Calculate correlation matrix`
`correlation_matrix = numerical_data.corr()`
`# Display correlation matrix`
`print("===== CORRELATION MATRIX =====")`
`print(correlation_matrix)`

	entity_id	internal_id	width	height
entity_id	1.000000	1.000000	-0.105263	-0.048043
internal_id	1.000000	1.000000	-0.105263	-0.048043
width	-0.105263	-0.105263	1.000000	0.624051
height	-0.048043	-0.048043	0.624051	1.000000

[1:]: